



UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Nombre de la Organización:

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO (UTEQ)

Presenta:

VICTOR GUADALUPE LOPEZ CEBREROS

Grupo:

IDGS17

Práctica 4. Herramientas para Administración de Logs. Parte 2 y 3.

SEGG-U2-P4H-2 y SEGG-U2-P4H-3

Enriquecimiento de Registros con Promtail y Loki, Consultas LogQL y Construcción de Dashboards

1. Datos generales

Campo	Valor
Aplicación monitoreada	VulnerableApp, rama secure, ejecución local en http://localhost:5277
Plataforma de observabilidad	Grafana, Loki y Promtail en contenedores Docker
Interfaz de Grafana	http://localhost:3000
Endpoint de ingesta de Loki	http://loki:3100/loki/api/v1/push
Puerto interno de Promtail	9080
Origen de los registros	VulnerableApp/Logs/*.txt mediante bind mount
Herramienta de comparación	Seq, desplegada en la práctica P3G-1

2. Objetivo

Enriquecer los registros producidos por la aplicación mediante etiquetas calculadas en el agente de recolección, sin modificar el código fuente, y explotar esa clasificación mediante consultas LogQL y dashboards persistentes orientados al monitoreo continuo y a la investigación de incidentes.

Ambas prácticas se documentan de forma conjunta por compartir la misma infraestructura y una única corrida de generación de actividad: P4H-2 configura la clasificación y P4H-3 la consume.

PARTE I — SEGG-U2-P4H-2

Integración y enriquecimiento con Promtail y Loki

3. Estructura del archivo de configuración de Promtail

Sección	Función
server	Configura el servidor HTTP interno de Promtail en el puerto 9080, empleado para exponer métricas y el endpoint de salud /ready. No recibe registros por esa vía.
clients	Lista de destinos hacia los que Promtail envía los registros ya procesados. En esta configuración existe un único cliente apuntando a Loki.
positions	Ruta del archivo positions.yaml, donde Promtail almacena, por cada archivo vigilado, el desplazamiento en bytes de la última línea leída.
scrape_configs	Conjunto de trabajos de recolección. Cada uno define qué archivos leer y cómo procesarlos mediante pipeline_stages.
static_configs	Dentro de un trabajo, define de forma fija —sin descubrimiento automático— el conjunto inicial de etiquetas y el patrón de rutas __path__ de los archivos a vigilar.
labels	Pares clave-valor adjuntos a cada línea que quedan indexados en Loki. Constituyen la base de toda consulta LogQL con la sintaxis {etiqueta="valor"}.

4. Monitoreo de múltiples archivos

Se configuraron dos trabajos de recolección:

Trabajo	Patrón de rutas	Propósito
vulnerableapp_logs	/var/log/vulnerableapp/log-*.txt	Archivo principal generado por Serilog
vulnerableapp_security_logs	/var/log/vulnerableapp/security-*.txt	Preparado para un futuro sink de

Trabajo

Patrón de rutas

Propósito

Serilog dedicado a seguridad

El segundo trabajo se declara aun cuando el archivo correspondiente no existe todavía. Promtail reevalúa periódicamente los patrones de ruta, de modo que el día en que ese archivo aparezca comenzará a recolectarse sin necesidad de modificar la configuración ni reiniciar el agente. Se trata de una decisión de diseño orientada a la evolución del sistema: la infraestructura anticipa el cambio en lugar de reaccionar a él.

4.1 Por qué Promtail requiere conocer la ubicación física de los archivos

La diferencia respecto de Serilog es de modelo de comunicación. Serilog **empuja** los eventos hacia sus destinos: conoce la dirección de Seq y transmite activamente hacia ella. Promtail opera en sentido inverso: es un agente que **extrae**, leyendo archivos de texto desde el sistema de archivos de manera análoga a tail -f.

VulnerableApp se ejecuta fuera de Docker y desconoce por completo la existencia de Promtail; no le envía nada ni mantiene con él conexión alguna. El único punto de contacto entre ambos es el sistema de archivos, expuesto dentro del contenedor mediante un bind mount. De ahí que Promtail necesite la ruta exacta dentro de su propio espacio de nombres para abrir y vigilar los archivos.

Esta arquitectura tiene una consecuencia notable: la aplicación puede detenerse, reiniciarse o incluso desinstalarse sin que la plataforma de observabilidad se vea afectada, y a la inversa. El acoplamiento entre ambos sistemas es prácticamente nulo.

5. Enriquecimiento mediante etiquetas

5.1 Concepto

Las etiquetas son metadatos clave-valor que Loki **indexa**, a diferencia del cuerpo de la línea, que se almacena comprimido sin indexar. Cada combinación única de etiquetas constituye un flujo independiente dentro del sistema.

5.2 Etiquetas configuradas

La clasificación se implementó mediante una etapa template dentro de pipeline_stages, que evalúa el contenido de cada mensaje con expresiones regulares y calcula dos campos:

Etiqueta	Valores	Criterio de asignación
application	Security	Mensajes de Auth.*, AuthEvent:* y verificación de hash BCrypt
	Audit	Mensajes de Comment.*, Api.* y consultas SQL
	VulnerableApp	Cualquier otro caso
module	authentication	Mismo criterio que Security
	orders	Operaciones CRUD sobre comentarios y API
	general	Resto de la actividad
environment	dev	Etiqueta estática declarada en static_configs
level	DBG, INF, WRN, ERR, FTL	Extraído del formato de salida de Serilog

Una etapa labels posterior promueve level, application y module de campos extraídos a etiquetas reales de Loki.

Cabe precisar que el valor orders no corresponde a un módulo de pedidos existente en la aplicación, sino que se adoptó como equivalente de dominio para las operaciones CRUD sobre comentarios y API, dado que la consigna de la práctica lo solicita de forma explícita.

5.3 Ventajas frente a la búsqueda textual

Loki indexa exclusivamente las etiquetas. Al consultar {application="Security"}, el motor localiza directamente los flujos pertinentes empleando ese índice, sin recorrer el contenido de las líneas. Una búsqueda de texto equivalente —mediante grep o un filtro de línea |= "Security"— exigiría escanear la totalidad del volumen almacenado.

La segunda ventaja, de naturaleza arquitectónica, resulta más significativa: las etiquetas permiten **clasificar sin modificar la aplicación**. La totalidad del conocimiento sobre qué constituye un evento de seguridad y qué uno de auditoría reside en la configuración de Promtail, no en el código C#. Un cambio en los criterios de clasificación se resuelve editando un archivo YAML y reiniciando un contenedor, sin recompilar, sin volver a probar y sin desplegar la aplicación.

6. Aplicación de la configuración

Tras modificar el archivo se reinició el agente y se verificó su carga correcta mediante la inspección de sus propios registros, donde deben aparecer los mensajes de recarga de configuración y de incorporación de destinos para ambos trabajos, sin líneas de nivel error.

Se confirmó posteriormente en Loki que los valores de las etiquetas quedaron correctamente acotados: application con tres valores, module con tres valores y level con cinco. Esta verificación es relevante porque una expresión regular defectuosa en la etapa template produciría etiquetas con valores espurios o vacíos, defecto que no genera error alguno y que solo se detecta inspeccionando los valores efectivamente registrados.

7. Actividad generada

Se ejercitó la aplicación con una secuencia diseñada para producir eventos de las tres categorías y de todos los niveles de severidad relevantes: navegación general, búsqueda, tres intentos consecutivos de autenticación fallida en menos de un minuto, autenticación exitosa, acceso al panel, alta de comentario válido y vacío, consultas a la API, excepción controlada, cierre de sesión y excepción no controlada.

La concentración temporal de los tres intentos fallidos fue deliberada, con el propósito de reproducir el patrón que se analiza en el caso de estudio de la sección 12.

8. Validación desde Grafana

Se ejecutaron las siguientes consultas en la vista exploratoria, verificando en cada caso que el conjunto de resultados correspondiera a la clasificación esperada:

logql

```
{job="vulnerableapp"}
```

```
{job="vulnerableapp", application="Security"}
```

```
{job="vulnerableapp", application="Audit"}
```

```
{job="vulnerableapp", module="orders"}
```

9. Reto P4H-2 — Etiquetas adicionales y filtrado por módulo

Procedimiento realizado:

1. Se agregó la etiqueta estática environment: dev en la sección static_configs de ambos trabajos de recolección.

2. Se incorporó una segunda etapa template, con destino en el campo module, que clasifica cada línea en authentication, orders o general aplicando la misma lógica de expresiones regulares empleada para application.
3. Se promovió module a etiqueta real mediante la etapa labels.
4. Se ejecutó en Grafana la consulta:

logql

```
{job="vulnerableapp", module="authentication"}
```

5. El resultado mostró exclusivamente los eventos de AuthController —autenticación, cierre de sesión, acceso al panel y verificación del hash—, excluyendo por completo los de comentarios y API, clasificados como orders, así como los generales.

Observación sobre la sintaxis. El valor de la etiqueta es authentication en minúsculas, conforme quedó definido en la plantilla, aunque la consigna lo mencione con inicial mayúscula. LogQL distingue entre mayúsculas y minúsculas en los valores de etiqueta, por lo que la consulta debe escribirse de forma exacta. Una discrepancia de este tipo no produce error sino un conjunto de resultados vacío, comportamiento que resulta engañoso durante la depuración.

10. Pregunta de reflexión P4H-2

¿Qué ocurriría si Promtail no almacenara la posición del último registro leído en positions.yaml? Explique cómo afectaría la recolección de eventos.

Sin positions.yaml, Promtail carecería de memoria sobre hasta qué punto ha leído cada archivo. Ante cualquier reinicio del contenedor —una recarga de configuración, una actualización de imagen o una simple caída— solo caben dos comportamientos posibles, y ambos resultan problemáticos.

Si retomara la lectura desde el inicio del archivo, generaría **eventos duplicados** en Loki. Las consecuencias exceden el desperdicio de almacenamiento: cualquier conteo, tasa o alerta basada en frecuencia quedaría distorsionada. Un panel de intentos de autenticación fallidos mostraría el doble de los reales, y una regla de alerta configurada sobre él se dispararía sin causa.

Si, en cambio, retomara la lectura desde el final del archivo en ese instante, **perdería** toda línea escrita durante el intervalo de inactividad del agente. Este escenario es el más grave desde la perspectiva de la seguridad, porque el hueco resultante es invisible: nada en Loki indica que faltan eventos, y una investigación posterior concluiría erróneamente que durante ese lapso no ocurrió nada. Un atacante consciente de esta característica podría incluso provocar el reinicio del agente para encubrir su actividad.

En este proyecto el archivo se conserva en un volumen con nombre que persiste a la recreación del contenedor, precisamente para evitar ambos escenarios. La existencia de este mecanismo ilustra un principio general de los sistemas de recolección: la garantía de entrega exige estado persistente, y ese estado debe sobrevivir al componente que lo produce.

11. Análisis comparativo P4H-2 — Seq frente a Grafana con Loki

Característica	Seq	Grafana + Loki
Búsqueda textual	Motor estructurado sobre propiedades de Serilog, sobre el texto ya acotado por muy cómodo para depurar un caso puntual.	LogQL con filtros de línea aplicados sobre el texto ya acotado por etiquetas. Exige aprender la sintaxis pero escala mejor.
Filtrado por etiquetas	No existe el concepto. Filtra por propiedades estructuradas del evento.	Es el mecanismo central: localiza flujos completos de forma indexada antes de examinar el texto.
Dashboards	Limitados. Concebido como visor y buscador de eventos.	Punto fuerte: paneles reutilizables, gráficas de tendencia, persistentes y compartibles.
Consultas avanzadas	Lenguaje propio orientado a propiedades de .NET.	Combina selección por etiqueta, filtro de texto y funciones de agregación.
Escalabilidad	Orientado a volúmenes moderados en desarrollo y preproducción.	Separa índice de etiquetas y almacenamiento de bloques comprimidos, diseñado para volúmenes considerablemente mayores.
Uso principal	Depuración puntual durante el desarrollo.	Observabilidad continua: tendencias, alertamiento y correlación entre módulos.

Observación derivada de la comparación práctica. Al buscar los mismos eventos en ambas herramientas, la diferencia operativa resultó evidente: en Seq fue necesario construir un filtro sobre texto o propiedades, mientras que en Grafana la selección se obtuvo directamente por etiqueta. La distinción no es únicamente de

comodidad, sino de coste computacional: la segunda opera sobre un índice, la primera sobre el contenido.

PARTE II — SEGG-U2-P4H-3

Consultas LogQL y construcción de dashboards

12. Sintaxis de LogQL

Consulta	Propósito	Resultado observado
{application="VulnerableApp"}	Selecciona por Eventos de página principal, etiqueta los flujos búsqueda y ciclo de vida, de actividad excluyendo lo clasificado general.	como seguridad o auditoría.
{application="Security"}	Filtra los eventos de autenticación fallida y exitosa, autenticación.	Registros de AuthController: cierre de sesión, accesos sin sesión.
{application="Audit"}	Filtra los eventos de auditoría.	Registros de CommentController y ApiController, junto con las consultas SQL que originan.
{job="vulnerableapp"} "Error"	Filtro de línea sobre = el texto, con independencia de las etiquetas.	Sí devuelve resultados , porque los mensajes de excepción están redactados en español y contienen literalmente esa palabra.
{job="vulnerableapp"} "Warning"	Mismo tipo de filtro, = buscando la español. La consulta palabra completa.	Cero resultados. Serilog emplea la abreviatura WRN y los mensajes están en español. La consulta equivalente válida es {job="vulnerableapp", level="WRN"}.

Consulta	Propósito	Resultado observado
<code>{application="Security"} "LoginFailed"</code>	Filtra primero por etiqueta indexada y Únicamente los intentos aplica después el fallidos, excluyendo cierre de filtro de texto sobre sesión y acceso al panel. el conjunto acotado.	

Las dos consultas centrales de esta tabla ilustran por qué la práctica exige documentar el propósito de cada una y explicar sus resultados. Un filtro de texto copiado de un ejemplo genérico puede devolver **cero resultados** simplemente por no coincidir con el idioma o el formato real de los registros del proyecto. Comprender qué produce efectivamente la aplicación es requisito previo a escribir cualquier filtro, y la ausencia de resultados no debe interpretarse de inmediato como ausencia de eventos.

13. Elementos de la vista exploratoria

Elemento	Función
Selector de origen de datos	Determina contra qué fuente se ejecuta la consulta. En esta práctica, Loki.
Editor de consultas	Área donde se escribe la expresión LogQL.
Selector de rango temporal	Acota la ventana de búsqueda. Constituye la causa más frecuente de resultados vacíos cuando la actividad se generó fuera del rango seleccionado.
Panel de resultados	Lista de líneas con su marca temporal y etiquetas asociadas. Cada línea se expande para mostrar el detalle completo y los campos detectados.

14. Consultas específicas

logql

Errores registrados

```
{job="vulnerableapp", level="ERR"}
```

Advertencias registradas

```
{job="vulnerableapp", level="WRN"}
```

Eventos de autenticación

```
{job="vulnerableapp", application="Security"}
```

Intentos fallidos dentro de autenticación

```
{job="vulnerableapp", application="Security"} |= "LoginFailed"
```

La progresión responde a una estrategia deliberada: se parte de una selección por etiqueta —indexada y rápida— y se refina mediante un filtro de texto aplicado únicamente sobre el conjunto ya acotado, en lugar de construir desde el inicio una expresión regular compleja que operaría sobre el volumen completo.

15. Caso de estudio — Análisis de un incidente

15.1 ¿Cuándo comenzaron los eventos?

logql

```
{job="vulnerableapp", module="authentication", level="WRN"}
```

Con el orden cronológico invertido para localizar el evento más antiguo, la primera advertencia de autenticación fallida apareció a las 16:49:38, y las siguientes se sucedieron hasta las 16:50:42. La concentración de varios intentos en poco más de un minuto corresponde al patrón característico de un ataque de fuerza bruta o de relleno de credenciales.

La densidad temporal es precisamente el indicador relevante. Un intento fallido aislado constituye ruido operativo normal —un usuario que se equivocó al teclear—; varios intentos concentrados contra cuentas distintas configuran una señal cualitativamente diferente.

15.2 ¿Qué módulo los generó?

No fue necesario inspeccionar el texto de los mensajes: la etiqueta `module="authentication"` lo indica de forma directa, y el hecho de que la consulta filtrada por ella devuelva resultados constituye la confirmación. Este es exactamente el beneficio que justificó configurar la etiqueta en P4H-2.

15.3 ¿Qué usuarios aparecen involucrados?

logql

```
{job="vulnerableapp", module="authentication", level="WRN"} | regex "Usuario:(?P<usuario>\\S+)"
```

Esta consulta introduce la **extracción en tiempo de consulta**, mecanismo que difiere del enriquecimiento realizado en Promtail. El nombre de usuario no se promovió a etiqueta —sería un valor de cardinalidad elevada— pero puede extraerse en el momento de consultar, obteniendo un campo utilizable para filtrado y agrupación sin coste alguno sobre el índice.

Los usuarios identificados fueron admin con tres intentos, root con dos y testuser con uno. Se registró adicionalmente un evento AuthEvent:AccesoNoAutorizado correspondiente a un usuario autenticado que intentó acceder a los datos de otro a través de la API.

La distribución resulta significativa por sí misma. Los nombres root y testuser no corresponden a cuentas existentes en el sistema, lo que descarta el error de tecleo de un usuario legítimo y sugiere un intento de descubrimiento contra nombres de cuenta comunes. Este matiz solo emerge al agrupar por usuario, y habría pasado inadvertido en la lectura secuencial de las líneas.

15.4 ¿Qué excepción se registró?

Las advertencias de autenticación fallida **no constituyen excepciones** del entorno de ejecución, sino advertencias de negocio derivadas de credenciales inválidas. La excepción real se produjo únicamente al activar el disparador de prueba:

```
logql
```

```
{job="vulnerableapp", level="ERR"} |= "Auth.Login"
```

La consulta localiza la línea de cabecera del evento de excepción no controlada.

Limitación identificada. El texto completo de la excepción, incluida su traza de pila, **no aparece con etiquetas propias** en Loki. La causa reside en que las líneas de continuación de una excepción no comienzan con marca temporal, y la expresión regular configurada en Promtail solo reconoce la primera línea de cada evento. Para consultar la traza completa es necesario abrir el archivo directamente o recurrir a Seq, que agrupa la excepción como un evento único.

Se trata de una limitación real de la configuración actual y merece consignarse en el análisis. Existe una técnica para resolverla —la etapa multiline de Promtail, que une las líneas de continuación en un solo evento antes de enviarlas— pero excede el alcance de esta práctica. La observación resulta relevante porque delimita con precisión qué preguntas puede responder esta plataforma y cuáles requieren acudir a otra fuente.

15.5 Evidencia que respalda las conclusiones

Las tres consultas anteriores con sus respectivos resultados, complementadas con la inspección directa del archivo de registro para obtener la traza de pila completa de la excepción.

16. Dashboards construidos

Ambos dashboards se definieron como archivos JSON versionables y se aprovisionan de forma automática al iniciar Grafana, en lugar de construirse manualmente desde la interfaz. Esta decisión los convierte en configuración reproducible: cualquier despliegue nuevo del entorno los recrea de manera idéntica, sin depender de la memoria de quien los armó ni del respaldo de la base de datos de Grafana.

16.1 Dashboard de monitoreo general

Panel	Tipo	Consulta LogQL	Justificación
Eventos por Serie nivel en el tiempo	por Serie	sum by (level) (count_over_time({job="vulnerableapp"}[\$__interval]))	Ofrece visión de tendencia y permite detectar un pico de errores o advertencias sin leer las líneas individualmente.
Errores	Lista de registros	{job="vulnerableapp", level="ERR"}	Acceso directo al detalle de cada error sin reescribir la consulta en cada revisión.
Autenticaciones	Lista de registros	{job="vulnerableapp", application="Security"}	Monitoreo centralizado de la

Panel	Tipo	Consulta LogQL	Justificación
			actividad de acceso.
Auditoría	Lista de registros	{job="vulnerableapp", application="Audit"}	Trazabilidad de operaciones CRUD con fines de cumplimiento.

El primer panel merece atención particular porque **transforma registros en métricas**. La función `count_over_time` convierte eventos discretos en una serie temporal agregada, y `sum by (level)` la descompone por severidad. El resultado ya no es una lista de líneas sino una gráfica sobre la cual pueden definirse umbrales y alertas: es el punto exacto donde el registro deja de ser un artefacto de depuración y pasa a ser un instrumento de monitoreo.

16.2 Dashboard de seguridad (Reto P4H-3)

Panel	Tipo	Consulta LogQL	Justificación
Tasa de intentos fallidos	Serie temporal	<code>sum(count_over_time({job="vulnerableapp", module="authentication", level="WRN"}[\$__interval]))</code>	Convierte eventos individuales en tendencia. Un salto brusco constituye la señal de un posible ataque de fuerza bruta.
Autenticaciones fallidas	Lista de registros	<code>{job="vulnerableapp", module="authentication", level="WRN"}</code>	Detalle por línea — usuario y dirección de origen — para

Panel	Tipo	Consulta LogQL	Justificación
			investigar una vez detectado el patrón.
Eventos de auditoría	Lista de registros	{job="vulnerableapp", application="Audit"}	Traza qué se creó o modificó y quién lo hizo, necesario tras un incidente.
Errores críticos	Lista de registros	{job="vulnerableapp", level="ERR"}	Visibilidad inmediata de fallas que requieren atención, con independencia del módulo.

La composición del dashboard responde a una lógica operativa deliberada: los paneles se ordenan de lo agregado a lo específico, replicando la secuencia natural de una investigación. La gráfica de tendencia responde a la pregunta «¿está ocurriendo algo?»; la lista de intentos fallidos responde a «¿qué exactamente?»; y los paneles de auditoría y errores responden a «¿qué consecuencias tuvo?».

17. Reto P4H-3 — Uso del dashboard en operación productiva

Fase de detección. El dashboard se mantendría visible en una pantalla de monitoreo o integrado al canal del equipo de guardia. El panel de tasa de intentos fallidos constituye un candidato directo a regla de alerta: al superar un umbral dentro de una ventana temporal breve, se dispara una notificación hacia el equipo de seguridad sin necesidad de supervisión humana continua.

La definición del umbral exige, no obstante, establecer previamente una línea base. Un valor demasiado bajo produce alertas espurias que conducen a la desatención sistemática de las notificaciones; uno demasiado alto permite que un ataque lento pase inadvertido. La determinación de ese valor requiere observar el comportamiento normal del sistema durante un período representativo, lo que constituye un argumento adicional a favor de desplegar la observabilidad antes de necesitarla.

Fase de respuesta. Los paneles de auditoría y de errores críticos operan una vez detectado el incidente, permitiendo reconstruir la línea temporal exacta de lo ocurrido sin necesidad de solicitar acceso a los archivos del servidor productivo. Todo queda centralizado y consultable desde Grafana bajo los mismos controles de acceso del resto de la plataforma.

Esta última consideración posee relevancia de seguridad propia. El acceso directo a los archivos de un servidor productivo constituye un privilegio elevado que conviene restringir; la consulta a través de Grafana permite conceder visibilidad sobre los registros sin otorgar acceso al sistema que los produce, materializando el principio de privilegio mínimo en el propio proceso de investigación.

18. Pregunta de reflexión P4H-3

¿Qué ventajas aporta LogQL frente a revisar directamente archivos de texto o realizar búsquedas simples en Seq?

La revisión directa de archivos obliga a emplear herramientas genéricas que desconocen por completo la estructura del registro. Cada búsqueda recorre el archivo carácter por carácter, no existe forma de expresar «únicamente los eventos de autenticación» sin depender de que una palabra concreta aparezca en el texto, y el enfoque deja de escalar en cuanto existen múltiples archivos rotando por fecha.

Seq mejora sustancialmente ese escenario, ya que comprende las propiedades estructuradas emitidas por Serilog y permite consultarlas como tales. Sin embargo, sigue operando sobre un almacén centralizado con un motor de búsqueda propio orientado al evento individual.

LogQL aporta dos ventajas concretas frente a ambos enfoques.

La primera es el **filtrado en dos fases**: la selección inicial por etiquetas indexadas y el filtro de texto posterior sobre un conjunto ya acotado. Nunca se examina contenido que la etiqueta permite descartar de antemano, lo que mantiene el rendimiento de las consultas incluso con volúmenes elevados.

La segunda, y de mayor alcance, es que **LogQL no es únicamente un buscador**. Sus funciones de agregación —`count_over_time`, `rate`, `sum by`— permiten transformar registros en métricas de tendencia, como la tasa de autenticaciones fallidas por minuto. Ni un archivo de texto ni Seq ofrecen esa capacidad de forma nativa. Es precisamente lo que habilita el tránsito desde «localizar un evento puntual» hacia «monitorear un sistema en producción» empleando un único lenguaje de consulta.

Conviene reconocer, en contrapartida, que LogQL exige un aprendizaje mayor y que la limitación identificada en la sección 15.4 —el tratamiento de excepciones

multilínea— evidencia que la superioridad no es absoluta. Ambas herramientas conservan su terreno propio.

19. Análisis comparativo P4H-3

Característica	Seq	Grafana + Loki
Búsqueda textual	Motor propio sobre propiedades estructuradas de Serilog.	Filtro por etiqueta indexada de filtros de línea sobre el texto.
Filtrado por etiquetas	Concepto inexistente; filtra por propiedades del evento.	Mecanismo central del sistema; determina qué flujos se consultan.
Dashboards	Visor de eventos, no concebido para paneles.	Paneles reutilizables, persistentes y combinables con gráficas de tendencia.
Investigación de incidentes	Superior para el caso puntual, gracias al agrupamiento automático de excepciones completas con su traza de pila.	Superior para correlacionar volumen y tendencia entre módulos; las excepciones multilínea requieren configuración adicional.
Uso principal	Depuración durante el desarrollo.	Observabilidad continua, alertamiento y monitoreo productivo.

Conclusión del comparativo. Seq y Grafana con Loki no compiten por el mismo caso de uso: se complementan. Seq resulta superior para la depuración inmediata durante el desarrollo, al agrupar una excepción con su traza completa como un evento único navegable, capacidad que en Loki exige configuración adicional. Grafana con Loki destaca cuando el volumen crece y se requiere indexar por categoría para evitar el escaneo de texto completo, construir paneles persistentes para monitoreo continuo y definir alertas sobre esas mismas consultas.

En un flujo de trabajo real, Seq acompaña al desarrollador en su equipo local y Grafana con Loki acompaña al sistema una vez que opera de forma continua. Esta práctica lo confirma de manera directa: ambas herramientas consumieron la misma fuente de registros de forma simultánea y sin interferencia alguna entre sí.

20. Conclusiones

Se enriquecieron los registros de la aplicación con cuatro etiquetas calculadas en el agente de recolección, sin modificar una sola línea del código fuente. La clasificación en dominios de seguridad, auditoría y operación general reside íntegramente en la configuración de Promtail, lo que permite modificarla sin recompilar ni desplegar la aplicación.

La distinción entre lo que se promueve a etiqueta y lo que se extrae en tiempo de consulta constituye la decisión de diseño central de la práctica. El nivel de severidad, la aplicación y el módulo poseen dominios acotados y resultan candidatos apropiados a etiqueta; el nombre de usuario y el identificador de correlación poseen cardinalidad elevada y deben extraerse en el momento de consultar, como se demostró en el caso de estudio.

El caso de estudio evidencia que el valor de la plataforma no reside en almacenar registros sino en permitir formular preguntas sobre ellos. La identificación de que dos de los usuarios atacados ni siquiera existen en el sistema —dato que transforma la interpretación del incidente— solo emergió al agrupar los eventos por usuario, operación inviable mediante lectura secuencial.

La limitación detectada en el tratamiento de excepciones multilínea aporta una conclusión de igual valor: conocer qué **no** puede responder una herramienta resulta tan importante como conocer sus capacidades. La coexistencia de Seq y Grafana sobre la misma fuente permitió, en este caso, cubrir el hueco de una con la fortaleza de la otra.

Finalmente, la conversión de registros en métricas de tendencia mediante funciones de agregación marca el límite entre depuración y monitoreo. Una lista de eventos responde a qué ocurrió; una serie temporal con umbral definido advierte de que algo está ocurriendo, y esa diferencia es la que separa la reacción de la detección.

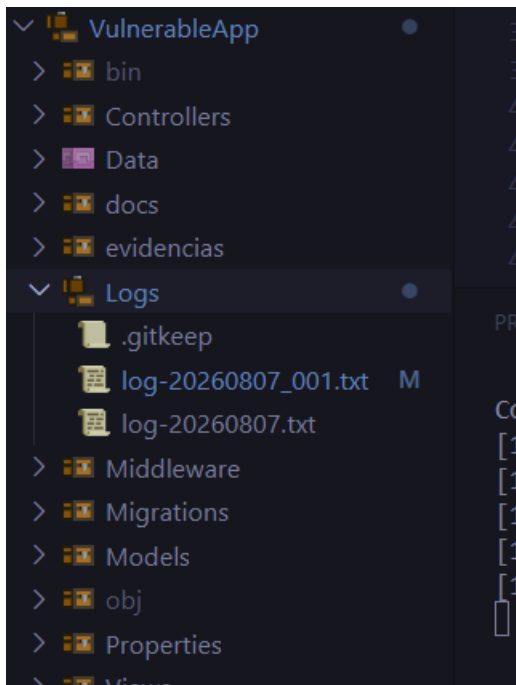
21. Referencias

- Grafana Labs. *LogQL* — Log stream selectors, line filters, parser expressions y funciones de agregación.
- Grafana Labs. *Loki* — Labels, cardinalidad y arquitectura de índice y bloques.
- Grafana Labs. *Promtail* — `scrape_configs`, `pipeline_stages`, `positions` y etapa multiline.
- Grafana Labs. *Provisioning* — Aprovisionamiento declarativo de dashboards y orígenes de datos.
- Grafana Labs. *Grafana Alerting* — Reglas sobre consultas y canales de notificación.
- OWASP. *Logging Cheat Sheet* — Monitoreo de eventos de autenticación.

22. Evidencias

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  PIXEL AGENTS

Compilando...
[17:15:54 INF] Iniciando VulnerableApp...
[17:15:54 INF] [cid:] Now listening on: http://localhost:5277
[17:15:55 INF] [cid:] Application started. Press Ctrl+C to shut down.
[17:15:55 INF] [cid:] Hosting environment: Development
[17:15:55 INF] [cid:] Content root path: C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp
█
```



```
C:\Users\victo\Documents\Práctica Creación de Solución>cd "observability-infra"

C:\Users\victo\Documents\Práctica Creación de Solución\observability-infra>docker compose ps
NAME          IMAGE          COMMAND          SERVICE    CREATED   STATUS    PORTS
grafana       grafana/grafana:10.4.3  "/run.sh"       grafana    36 minutes ago  Up 36 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
loki          grafana/loki:2.9.8    "/usr/bin/loki -conf..."  loki      36 minutes ago  Up 36 minutes  0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp
promtail      grafana/promtail:2.9.8  "/usr/bin/promtail -..."  promtail  36 minutes ago  Up 25 minutes

C:\Users\victo\Documents\Práctica Creación de Solución\observability-infra>█
```

localhost:5341/#/events?range=1d

Render Dashboard

Seq Personal Find commands, shortcuts, and screens Ctrl K Events Metrics Dashboards

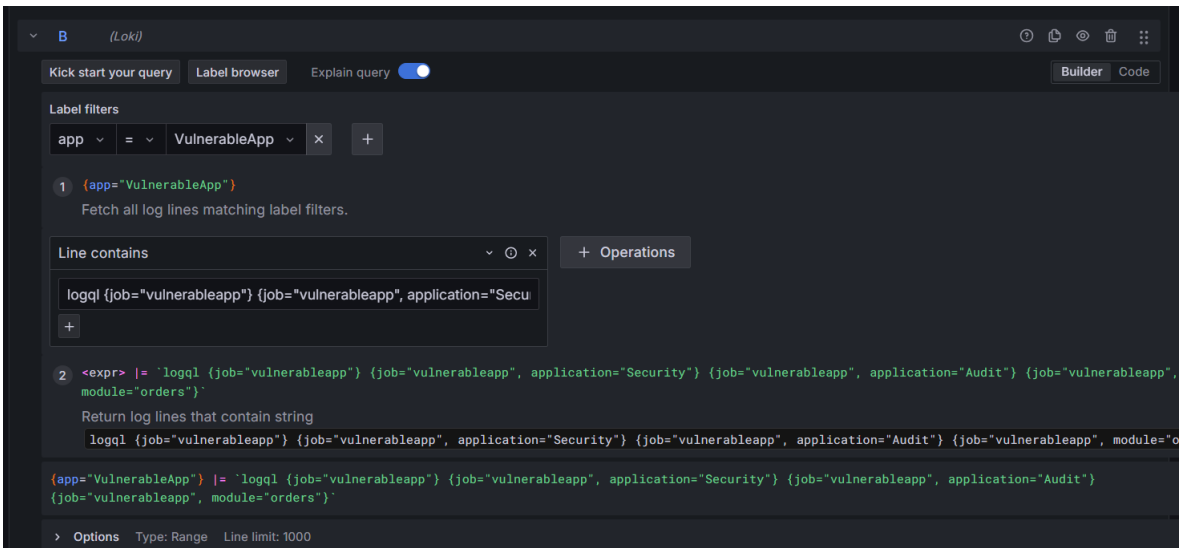
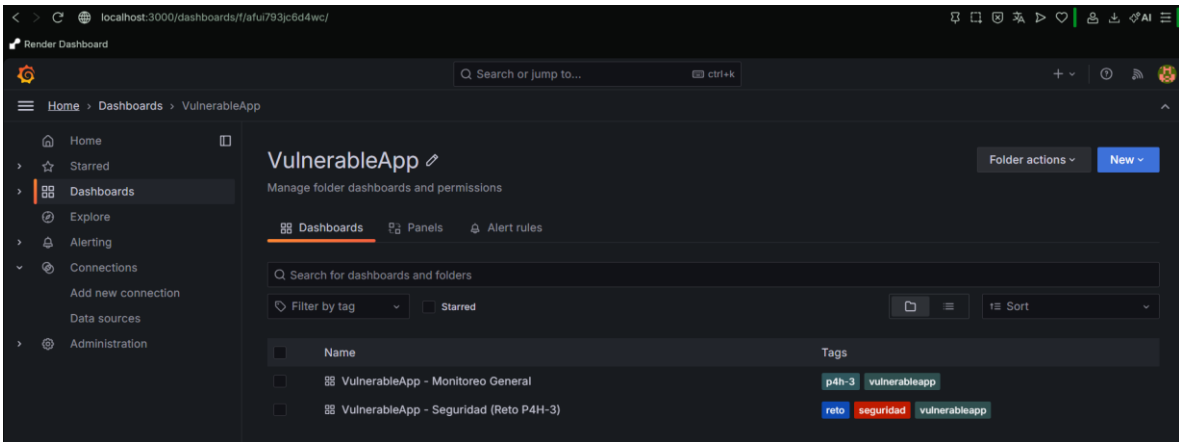
Last 1d

07 Aug 2026 17:18:55.373	HTTP GET /favicon.ico respondio 200 en 14.7982 ms
07 Aug 2026 17:18:54.512	HTTP GET /lib/jquery/dist/jquery.min.js respondio 200 en 212.6302 ms
07 Aug 2026 17:18:54.384	HTTP GET /lib/bootstrap/dist/js/bootstrap.bundle.min.js respondio 200 en 239.0538 ms
07 Aug 2026 17:18:54.380	HTTP GET /js/site.js respondio 200 en 122.3424 ms
07 Aug 2026 17:18:54.161	HTTP GET /lib/bootstrap/dist/css/bootstrap.min.css respondio 200 en 327.0248 ms
07 Aug 2026 17:18:54.002	HTTP GET /VulnerableApp.styles.css respondio 200 en 167.4459 ms
07 Aug 2026 17:18:54.002	HTTP GET /css/site.css respondio 200 en 169.1595 ms
07 Aug 2026 17:18:53.700	HTTP GET / respondio 200 en 2913.2685 ms
07 Aug 2026 17:18:52.470	Fin Home.Index Usuario:anonimo DuracionMs:72
07 Aug 2026 17:18:52.442	Inicio Home.Index Usuario:anonimo IP:::1
07 Aug 2026 17:18:50.632	Failed to determine the https port for redirect.
07 Aug 2026 17:15:55.012	Content root path: C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp
07 Aug 2026 17:15:55.011	Hosting environment: Development
07 Aug 2026 17:15:55.009	Application started. Press Ctrl+C to shut down.
07 Aug 2026 17:15:54.957	Now listening on: http://localhost:5277
07 Aug 2026 16:50:44.678	HTTP POST /Auth/Login respondio 500 en 3.3818 ms
07 Aug 2026 16:50:44.677	Unhandled exception. Metodo:POST Ruta:/Auth/Login CorrelationId:e458d343-4168-4baf-b81c-ded23bd3e5dd
07 Aug 2026 16:50:44.676	Inicio Auth.Login (POST) Usuario:_UNCONTROLLED_EXCEPTION_TEST_ IP:::1
07 Aug 2026 16:50:43.489	HTTP GET /Auth/Logout respondio 302 en 2.1164 ms
07 Aug 2026 16:50:43.488	AuthEvent:Logout Usuario:admin IP:::1
07 Aug 2026 16:50:42.323	HTTP GET /Search respondio 200 en 3.3253 ms
07 Aug 2026 16:50:42.322	Excepcion controlada capturada en Search.Index Usuario:anonimo IP:::1
07 Aug 2026 16:50:42.322	Inicio Search.Index Usuario:anonimo IP:::1 Parametros:{search: '_CONTROLLED_EXCEPTION_TEST_'}
07 Aug 2026 16:50:42.237	HTTP GET /api/user/2 respondio 403 en 4.6137 ms
07 Aug 2026 16:50:42.235	AuthEvent:AccesoNoAutorizado UsuarioId:1 intento acceder a UsuarioId:2 IP:::1
07 Aug 2026 16:50:42.234	Inicio Api.GetUser IP:::1 Parametros:{id: 2}
07 Aug 2026 16:50:42.165	HTTP GET /api/user/1 respondio 200 en 6.7001 ms
07 Aug 2026 16:50:42.165	Fin Api.GetUser UsuarioId:1 DuracionMs:5
07 Aug 2026 16:50:42.160	Inicio Api.GetUser IP:::1 Parametros:{id: 1}

```
C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp>dotnet run
Usando la configuración de inicio de C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp\Properties\launchSettings.json...
Compilando...
[17:15:54 INF] Iniciando VulnerableApp...
[17:15:54 INF] [cid:] Now listening on: http://localhost:5277
[17:15:55 INF] [cid:] Application started. Press Ctrl+C to shut down.
[17:15:55 INF] [cid:] Hosting environment: Development
[17:15:55 INF] [cid:] Content root path: C:\Users\victo\Documents\Práctica Creación de Solución\VulnerableApp
[17:18:50 WRN] [cid:fdc12c20-cc0d-4eba-b519-e6e9c8e53b45] Failed to determine the https port for redirect.
[17:18:52 INF] [cid:fdc12c20-cc0d-4eba-b519-e6e9c8e53b45] Inicio Home.Index Usuario:anonimo IP:::1
[17:18:52 INF] [cid:fdc12c20-cc0d-4eba-b519-e6e9c8e53b45] Fin Home.Index Usuario:anonimo DuracionMs:72
[17:18:53 INF] [cid:fdc12c20-cc0d-4eba-b519-e6e9c8e53b45] HTTP GET / respondio 200 en 2913.2685 ms
[17:18:54 INF] [cid:c640825b-975d-41f4-9127-454be43d928b] HTTP GET /VulnerableApp.styles.css respondio 200 en 167.4459 ms
[17:18:54 INF] [cid:278699e6-5ec0-4bae-aa93-01045697f70c] HTTP GET /css/site.css respondio 200 en 169.1595 ms
[17:18:54 INF] [cid:7e834c36-d1af-4009-a650-c038df272858] HTTP GET /lib/bootstrap/dist/css/bootstrap.min.css respondio 200 en 327.0248 ms
[17:18:54 INF] [cid:63b96701-b536-46f0-b6ce-3750e5c95c22] HTTP GET /js/site.js respondio 200 en 122.3424 ms
[17:18:54 INF] [cid:6a47feff-f2a6-4fbc-a9cb-cdb93e721ebc] HTTP GET /lib/bootstrap/dist/js/bootstrap.bundle.min.js respondio 200 en 239.0538 ms
[17:18:54 INF] [cid:2b4d23e2-1748-4536-a46a-65c96f3494e1] HTTP GET /lib/jquery/dist/jquery.min.js respondio 200 en 212.6302 ms
[17:18:55 INF] [cid:ffb6bef0-794c-41f9-9701-0793e1601116] HTTP GET /favicon.ico respondio 200 en 14.7982 ms
[]
```

```
C:\Users\victo\Documents\Práctica Creación de Solución\observability-infra>docker compose restart promtail
[+] restart 0/1
- Container promtail Restarting 2.2s

C:\Users\victo\Documents\Práctica Creación de Solución\observability-infra>docker compose logs promtail --tail 30
promtail | level=info ts=2026-08-07T22:41:11.136737902Z caller=promtail.go:133 msg="Reloading configuration file" md5sum=cc9f61f166b56419e1f231d6f3911977
promtail | level=info ts=2026-08-07T22:41:11.150113168Z caller=server.go:322 http[::]:9080 grpc[::]:36941 msg="server listening on addresses"
promtail | level=info ts=2026-08-07T22:41:11.153974636Z caller=main.go:174 msg="Starting Promtail" version="(version=2.9.8, branch=HEAD, revision=94e00299ec)"
promtail | level=warn ts=2026-08-07T22:41:11.154136107Z caller=promtail.go:263 msg="enable watchConfig"
promtail | level=info ts=2026-08-07T22:41:16.148838175Z caller=filetargetmanager.go:361 msg="Adding target" key="/var/log/vulnerableapp/log-*.txt:{app=\"VulnerableApp\", environment=\"dev\", job=\"vulnerableapp\"}"
promtail | level=info ts=2026-08-07T22:41:16.148940555Z caller=filetargetmanager.go:361 msg="Adding target" key="/var/log/vulnerableapp/security-*.txt:{app=\"VulnerableApp\", application=\"Security\", environment=\"dev\", job=\"vulnerableapp\", module=\"authentication\"}"
promtail | level=info ts=2026-08-07T22:41:16.171477273Z caller=filetarget.go:313 msg="watching new directory" directory=/var/log/vulnerableapp
promtail | level=info ts=2026-08-07T22:41:16.17935539Z caller=tailer.go:145 component=tailer msg="tail routine: started" path=/var/log/vulnerableapp/log-20260807.txt
promtail | ts=2026-08-07T22:41:16.181373702Z caller=log.go:168 level=info msg="Seeked /var/log/vulnerableapp/log-20260807.txt - &{Offset:0 Whence:0}"
promtail | ts=2026-08-07T22:41:16.18588514Z caller=log.go:168 level=info msg="Seeked /var/log/vulnerableapp/log-20260807_001.txt - &{Offset:0 Whence:0}"
promtail | level=info ts=2026-08-07T22:41:16.18570517Z caller=tailer.go:145 component=tailer msg="tail routine: started" path=/var/log/vulnerableapp/log-20260807_001.txt
promtail | level=info ts=2026-08-07T22:52:22.307734745Z caller=promtail.go:133 msg="Reloading configuration file" md5sum=572362b7cc7abe848d9f63a47ecfb4ff
promtail | level=info ts=2026-08-07T22:52:22.324794374Z caller=server.go:322 http[::]:9080 grpc[::]:46103 msg="server listening on addresses"
promtail | level=info ts=2026-08-07T22:52:22.328367348Z caller=main.go:174 msg="Starting Promtail" version="(version=2.9.8, branch=HEAD, revision=94e00299ec)"
promtail | level=warn ts=2026-08-07T22:52:22.328878012Z caller=promtail.go:263 msg="enable watchConfig"
promtail | level=info ts=2026-08-07T22:52:27.31851321Z caller=filetargetmanager.go:361 msg="Adding target" key="/var/log/vulnerableapp/log-*.txt:{app=\"VulnerableApp\", environment=\"dev\", job=\"vulnerableapp\"}"
promtail | level=info ts=2026-08-07T22:52:27.319103964Z caller=filetargetmanager.go:361 msg="Adding target" key="/var/log/vulnerableapp/security-*.txt:{app=\"VulnerableApp\", application=\"Security\", environment=\"dev\", job=\"vulnerableapp\", module=\"authentication\"}"
promtail | level=info ts=2026-08-07T22:52:27.337720576Z caller=filetarget.go:313 msg="watching new directory" directory=/var/log/vulnerableapp
promtail | level=info ts=2026-08-07T22:52:27.347331873Z caller=tailer.go:145 component=tailer msg="tail routine: started" path=/var/log/vulnerableapp/log-20260807.txt
```



▼ B (Loki)

Kick start your query Label browser Explain query ☒ Builder Code

Label filters

app = VulnerableApp × +

1 {app="VulnerableApp"}
Fetch all log lines matching label filters.

Line contains × + Operations

{job="vulnerableapp", application="Security"}
+

2 <expr> |= `{{job="vulnerableapp", application="Security"}}`
Return log lines that contain string {{job="vulnerableapp", application="Security"}}.

{app="VulnerableApp"} |= `{{job="vulnerableapp", application="Security"}}`

> Options Type: Range Line limit: 1000

observability-infra > docker-compose.yml

Run All Services | You, 1 hour ago | 1 author (You)

```
1 services:
2
3   Run Service
4   grafana:
5     image: grafana/grafana:10.4.3
6     container_name: grafana
7     ports:
8       - "3000:3000"
9     environment:
10      - GF_SECURITY_ADMIN_USER=admin
11      - GF_SECURITY_ADMIN_PASSWORD=admin
12      - GF_AUTH_ANONYMOUS_ENABLED=false
13     volumes:
14      - grafana_data:/var/lib/grafana
15      - ./grafana/provisioning:/etc/grafana/provisioning:ro
16     depends_on:
17      - loki
18     networks:
19      - observability
20
21   Run Service
22   loki:
23     image: grafana/loki:2.9.8
24     container_name: loki
25     ports:
26       - "3100:3100"
27     command: -config.file=/etc/loki/local-config.yaml
28     volumes:
29      - loki_data:/loki
30     networks:
31      - observability
32
33   Run Service
34   promtail:      You, 1 hour ago • Arreglando VulnerableApp 🐛
35     image: grafana/promtail:2.9.8
36     container_name: promtail
37     volumes:
```